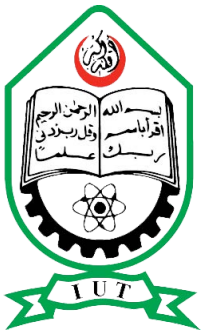
ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

ORGANISATION OF ISLAMIC COOPERATION (OIC)

Department of Computer Science and Engineering (CSE)



Project Proposal Report

SWE 4404: Software Project Lab II

Project Title

WanderVille

Name of the Supervisor

Md. Tariquzzaman

Team Members

Safia Aman Anika	230042120
Musfeka Ferdousi	230042126
Khadiza Sultana	230042125

Table of Contents

1. Project Overview 3
2. Motivation 4
3. User Requirement Analysis 4
4. Flow Chart and Class Diagram. 8
5. Key Features and Use Cases 10
6. Tools and Technologies 11
7. Proposed Project Timeline 12
8. Action Taken Based on Feedback 12
9. Conclusion 13

1. Project Overview

Wanderville is a 2D pixelated cozy village game for desktop users. A player signs up or logs in, enters a small persistent village, moves around the map, and performs peaceful activities such as farming, fishing, inventory use, pet interaction, and social interaction.

The project is designed as a real-time multiplayer architecture system. The Godot Engine client renders the game world and handles player interaction, while the FastAPI backend manages authentication, game logic, and server-side rules. Persistent data, such as player progress and world state, is stored in SQLite.

A WebSocket-based communication layer will enable real-time interaction between the client and server, supporting live updates and synchronized gameplay. The initial focus is to build a stable, single-player-like village experience powered by real accounts and persistent saves. Once this foundation is solid, multiplayer features and proximity chat will be introduced on top of the existing real-time system.

1.1. Main Objectives

- Build a playable cozy village prototype with a small isometric map.
- Implement real signup, login, password hashing, and token-based access.
- Design and implement a relational database schema covering players, inventory, farming, fishing, social relationships, and world state.
- Real-time events will use WebSocket, while authentication and data queries will use REST endpoints.
- Use a clean client-server architecture suitable for team development.
- Add friend, gift, shared community, and proximity chat features gradually.

2. Motivation

Cozy games are popular because they offer relaxation, collection, creativity, and social connection without combat pressure. However, many student game projects only focus on visible gameplay and do not show strong backend engineering. Wanderville is motivated by the idea of combining a calm game experience with software engineering depth.

Attributes	How Wanderville Works
Relaxing gameplay	Farming, fishing, pets, housing, and seasonal world activities.
Saved progress	Account-based persistent data through FastAPI and SQLite.
Social play	Friend requests, gifting, community area, and later proximity chat.
Technical depth	Authentication, authorization, APIs, database models, tests, and WebSocket plan.
Manageable scope	Build a stable V1 first; add multiplayer after the core systems are working.

3. User Requirement Analysis

3.1. Target Users

The target users are casual desktop game players, especially students, young adults, or corporate workers who enjoy peaceful games, simple controls, cute visuals, and light social interaction after a long, hard day.

3.2. Real-Life Stakeholder

3.2.1. Stakeholder:

A university student who plays casual games during study breaks and prefers relaxing, non-combat gameplay. Their needs include easy signup and login, a saved profile, a clear map, simple controls, a readable interface, relaxing activities, pet interaction, and a way to connect with friends without stress.

3.2.2. Stakeholder:

An office worker who unwinds after long work hours by playing low-effort casual games. Their needs include a quick and simple login, a saved profile that remembers their progress, a visually calm environment, intuitive controls that require no learning curve, relaxing activities such as farming and fishing, a pet to interact with, and a way to stay connected with friends without any competitive pressure or time commitment.

3.3 User Requirements

ID	Requirement
UR-01	A player shall be able to create an account and log in to access the game.
UR-02	A player shall be able to move around the village and see their character on the map.
UR-03	A player shall be able to farm crops and collect the harvest.
UR-04	A player shall be able to fish and collect caught fish.
UR-05	A player shall be able to view and manage their inventory.
UR-06	A player shall be able to interact with a pet companion.
UR-07	A player shall be able to add friends and send gifts.
UR-08	A player shall experience a world that changes with seasons, time of day, and weather.

3.3 System Requirements

3.3.1 Functional Requirements

ID	Requirement
FR-01	The system shall allow a player to register a new account using a unique username, password, and display name.
FR-02	The system shall allow a registered player to log in using their username and password and receive a secure access token.
FR-03	The system shall store all passwords as hashed values and shall never store plain-text passwords.
FR-04	The system shall allow a logged-in player to enter the main village world and display their name above their character.
FR-05	The system shall allow a player to move their character across the isometric village map using keyboard input.
FR-06	The system shall allow a player to plant a seed on an empty farm plot.
FR-07	The system shall allow a player to water a planted crop to advance its growth.
FR-08	The system shall allow a player to harvest a fully grown crop and add it to their inventory.
FR-09	The system shall allow a player to cast a fishing rod in a fish region and receive a caught item in their inventory.
FR-10	The system shall allow a player to view all items in their inventory, including quantity and item type.
FR-11	The system shall allow a player to interact with their pet companion, and the pet shall display visual emotional states (e.g., happy, sad, idle).
FR-12	The system shall allow a player to send a friend request to another player by username.

FR-13	The system shall allow a player to send a gift item from their inventory to a friend.
FR-14	The system shall allow a player to claim a received gift, which adds the item to their inventory.
FR-15	The system shall broadcast world state changes (season, time of day, weather) to all connected players in real time via WebSocket.
FR-16	The system shall notify a player via WebSocket when their crop is ready to harvest.
FR-17	The system shall reject any API request that does not include a valid access token.

3.3.2 Non-Functional Requirements

ID	Requirement
NFR-01	The system shall allow a first-time player to complete signup and enter the game world within 3 minutes without external assistance.
NFR-02	The system shall process a login request and return an access token within 2 seconds under normal load.
NFR-03	The system shall deliver WebSocket world state updates to all connected clients within 1 second of a server-side change.
NFR-04	The database schema shall be version-controlled using migration files so that all team members can reproduce an identical local setup.
NFR-05	The backend shall be structured in separate modules (authentication, farming, fishing, social, world state) so that each module can be tested independently.

4. Flow Chart and Class Diagram

4.1. User Flow Chart

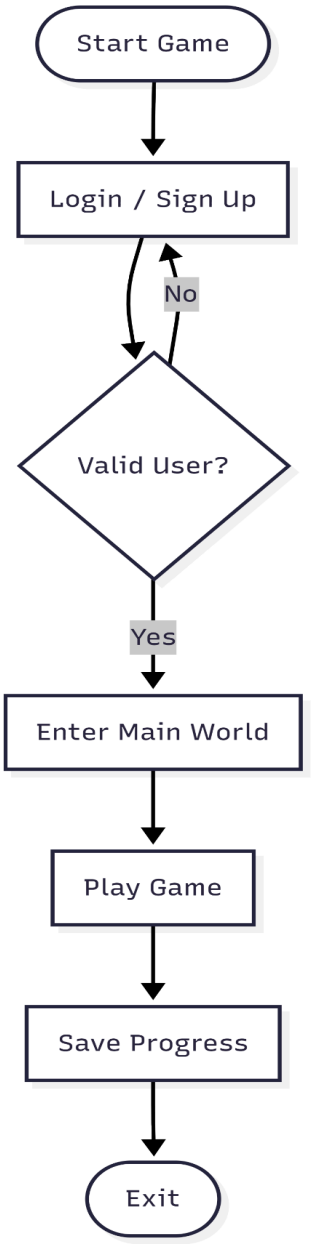


Figure 1. Basic user flow

4.2. ER Diagram

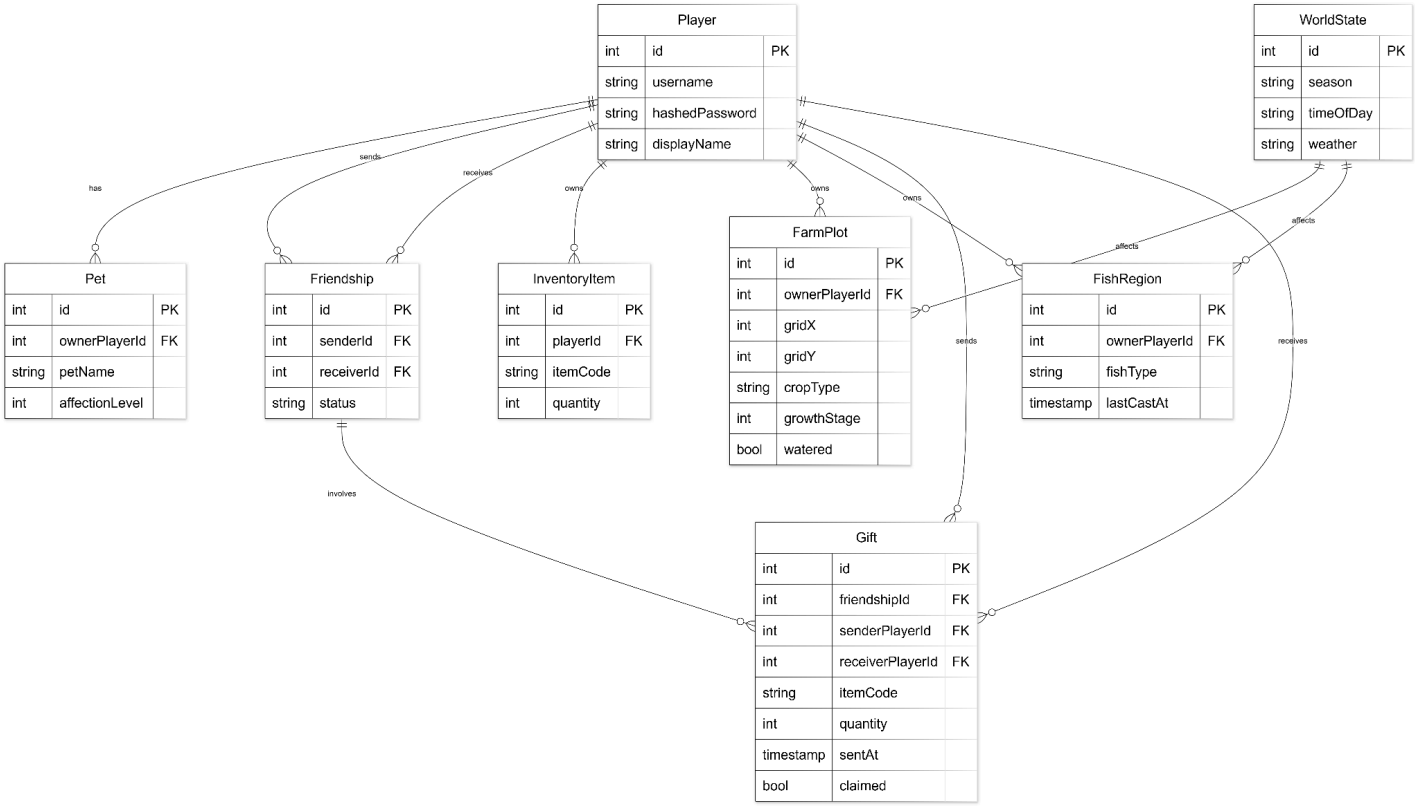


Figure 2. ER diagram

5. Key Features and Use Cases

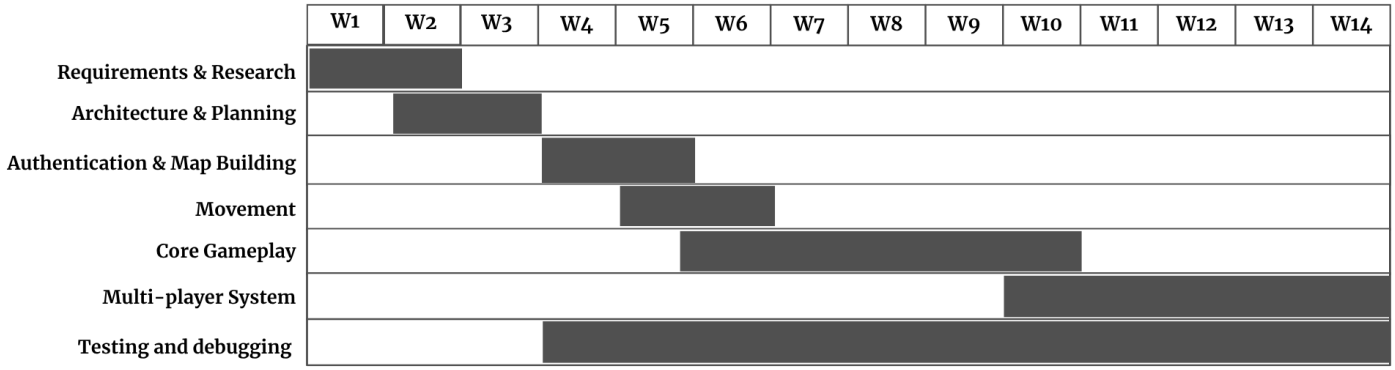
Feature	Use Case Description
Authentication	A player shall create an account, log in with their credentials, receive a secure token, and use it to access all protected game features.
Isometric main world	A player shall enter a persistent village map, move their character using keyboard controls, and see their display name shown above their character.
Farming system	A player shall plant seeds on their farm plot, water crops to advance growth, and harvest fully grown crops. The server shall verify plot ownership and update the player's inventory upon harvest.
Fishing system	A player shall cast a fishing rod in a designated fish region and receive a caught item added directly to their inventory. The server shall determine the catch based on the current world state.
Inventory system	A player shall view all items collected from farming, fishing, and received gifts, including item type and quantity.
Pet companion	A player shall interact with a pet companion that follows their character and displays simple emotional reactions such as happy, sleepy, excited, or surprised based on interaction.
World and housing system	A player shall start with a basic home structure. The world shall reflect seasonal changes, day and night cycles, and weather conditions that affect the visual environment and gameplay.

Friends and gifts	A player shall search for other players by username, send friend requests, and send inventory items as gifts to players with accepted friendship status.
Community area	Logged-in players shall join a shared community space where character movement and proximity-based text chat are synchronized in real time via WebSocket.

6. Tools and Technologies

Tool	Purpose
Godot 4.4 & GDScript	Desktop game client, scenes, UI, movement, maps, and animation.
FastAPI and Python	Backend API, authentication, services, and later WebSocket.
SQLite	Lightweight local development database.
SQLAlchemy	Database models, sessions, and table creation through Python.
JWT & bcrypt/passlib	Token-based login and password hashing.
WebSocket	Real-time movement and proximity chat in the community area.
APScheduler	Scheduled world events such as crop growth or day/night changes.
GitHub and VS Code	Collaboration, branching, code review, and development workflow.
Pytest	Backend unit and integration tests.

7. Proposed Project Timeline



8. Action Taken Based on Feedback

Feedback	Action taken
Do not start with full multiplayer	Development order has been restructured to begin with login, database setup, main world, player movement, and core gameplay before any multiplayer features.
Make the project manageable	V1 has been scoped down to a small backend-supported village experience rather than a full online world, with multiplayer features planned for V2.
Make team responsibilities clearer	Responsibilities have been formally divided: one member handles the Godot client and UI, one handles backend API and integration, and one handles database design and gameplay logic.
Ensure the project remains deliverable within the semester timeframe	A feature priority order has been established. If time constraints arise, lower-priority features such as fishing, the gifting system, and multiplayer community features will be descoped from V1 and deferred to V2, ensuring the core loop of login, farming, inventory, and movement is always delivered.

Reduce art production burden on the team	Free and openly licensed pixel-art assets from sources such as the Godot Asset Library and itch.io will be used as the visual foundation, allowing the team to focus development effort on backend logic and gameplay systems rather than original asset production.

9. Conclusion

Wanderville combines a cozy village game experience, which covers authentication, API design, database modeling, client-server architecture, and real-time communication. Free and openly licensed pixel-art assets will be used so the team can focus on building and integrating systems correctly. If time constraints arise, lower-priority features will be deferred to V2, ensuring the core gameplay loop is delivered within the semester. The project is designed to start small, build correctly, and expand only when the foundation is stable.